PLACEMENT PREP AI PLATFORM

Project Presentation Document

1. PROJECT OVERVIEW

Project Name: PlacementPrep AI - An Intelligent Platform for Campus Placement Preparation

Technology Stack: React.js, JavaScript, Tailwind CSS, Browser APIs (Web Speech, localStorage, getUserMedia, File API)

Project Type: Single Page Application (SPA) - Client-side only, no backend server

Purpose: Help students prepare for campus placements through mock interviews with simulated facial expression recognition, communication practice, skill-based tests, and AI-matched job recommendations.

2. KEY FEATURES IMPLEMENTED

Feature 1: Mock Interview with Simulated FER (Facial Expression Recognition)

What it does:

- Opens webcam using getUserMedia API and displays video feed
- Simulates emotion detection (Happy, Sad, Angry, Fear, Surprise, Neutral, Disgust)
- Updates emotion data every 3 seconds using setInterval
- Tracks dominant emotion and calculates trends (improving/declining/stable)
- Presents 10 HR/behavioral questions with timer (90-180 seconds each)

How it works:

```
1. getUserMedia API → Opens camera
2. Math.random() → Generates emotion percentages (NOT real AI)
3. JavaScript logic → Normalizes to 100%, finds dominant emotion
4. React state → Updates UI in real-time
5. Interview report saved to localStorage
```

Feature 2: AI Resume Analysis & Personalized Content Generation

What it does:

- User uploads PDF resume using File API

- Simulates skill extraction (randomly selects 5-12 skills from 40+ predefined skills)
- Determines career path based on skill clustering (Frontend/Backend/Full Stack/Data Science/Mobile/DevOps)
- Identifies skill gaps by comparing against desired skills
- Generates personalized content: 10 interview questions, 15-30 study resources with real URLs, 15-30 test questions

How it works:

```
1. File API → Captures PDF file (doesn't actually parse it)
2. Math.random() → Picks 5-12 random skills from array
3. if/else logic → Determines career path based on skills
4. Array.filter() → Matches jobs/courses to detected skills
5. Template strings → Generates personalized questions
6. localStorage → Saves all results with user ID
```

Feature 3: Communication Skills Practice with Voice Recognition

What it does:

- 4 activity types: Read Aloud, Jumbled Sentences, Listening Exercise, Speaking Practice
- Real-time speech-to-text conversion using Web Speech API
- Scores each activity and tracks progress

How it works:

```
1. Web Speech API (webkitSpeechRecognition) → Captures voice
2. Browser converts speech to text automatically
3. JavaScript compares user response to correct answer
4. Generates score based on response length and accuracy
```

Feature 4: Skill-Based Mock Tests with Analytics

What it does:

- 25 pre-defined multiple-choice questions across 8 categories (DSA, Database, Web Dev, OOP, etc.)
- Filter by category and difficulty (Easy/Medium/Hard)
- Timed tests with countdown timer
- Detailed results with explanations and analytics dashboard

How it works:

```
1. Array.filter() → Filters questions by category/difficulty
2. setInterval() → Countdown timer (updates every second)
3. Object to store answers {questionId: selectedOption}
4. Loop through answers, compare with correct index
5. Calculate percentage, save to localStorage
```

Feature 5: AI-Powered Job Matching Portal

What it does:

- 29 job listings (14 full-time jobs + 15 internships) across multiple tech domains
- Matches jobs to user skills with percentage match score
- Search and filter by location, salary, skills, job type
- One-click application tracking

How it works:

```
1. Array intersection → Finds matching skills between user and job
2. Calculate match score = (matching skills / total skills) * 100
3. Array.sort() → Ranks jobs by match score
4. localStorage → Tracks applied jobs per user
```

3. DATA STRUCTURES USED

All data is stored in JavaScript Arrays (not databases):

- Skills Array - 40+ technical skills across 7 categories
- Course Catalog Array - 24 online courses with pricing and skills
- Job Database Array - 29 positions with requirements and salaries
- Question Bank Array - 25 MCQs across 8 technical categories
- Interview Question Array - 10 HR/behavioral questions
- Communication Activities Array - 4 exercise types with practice content

4. TECHNICAL ARCHITECTURE

Frontend: React.js v18.2 with functional components and hooks

Styling: Tailwind CSS v3 for responsive design

State Management: React useState and useEffect hooks

Data Persistence: Browser localStorage API (5-10MB client-side storage)

Browser APIs Used:

- Web Speech API - For voice recognition in communication practice
- getUserMedia API - For webcam access in mock interviews
- File API - For PDF resume upload
- localStorage API - For persistent data storage

5. IMPORTANT: WHAT'S SIMULATED VS REAL

✓ REAL / WORKING:

- React frontend with responsive UI
- Web Speech API for real voice recognition
- Camera access and video display
- PDF file upload functionality
- Timed tests and scoring system
- Progress tracking and analytics

□ SIMULATED (Proof-of-Concept):

- *Facial Expression Recognition - Uses Math.random() to generate emotion data, NOT actual computer vision*

- *Resume Parsing - Randomly selects skills from predefined array, does NOT actually read PDF content*
- *AI Content Generation - Uses rule-based templates, NOT machine learning*

✗ NOT IMPLEMENTED:

- No TensorFlow.js or trained ML models
- No NLP libraries or text processing
- No backend server (Node.js/Python)
- No database (MongoDB/Firebase)
- No actual AI-powered features

6. PROJECT STATISTICS

- Total Lines of Code: ~3,500 lines (across 9 files)
- React Components: 1 main component with multiple render functions
- State Variables: 50+ useState hooks
- Data Arrays: 6 major datasets (skills, courses, jobs, questions)
- Main Features: 5 core features fully functional
- Browser APIs Used: 4 (Web Speech, getUserMedia, File API, localStorage)

7. CONCLUSION & FUTURE SCOPE

Current Status:

This is a fully functional client-side React application that demonstrates the UI/UX and workflow of an AI-powered placement preparation platform. While FER and resume parsing are simulated, all other features (voice recognition, tests, job matching, progress tracking) work perfectly.

Future Enhancements:

- Integrate TensorFlow.js for real facial expression recognition
- Add PDF parsing library (pdf-parse) for actual resume text extraction
- Implement backend server (Node.js/Express) for data persistence
- Connect to MongoDB/Firebase for scalable database
- Add NLP libraries for better skill extraction and content matching